

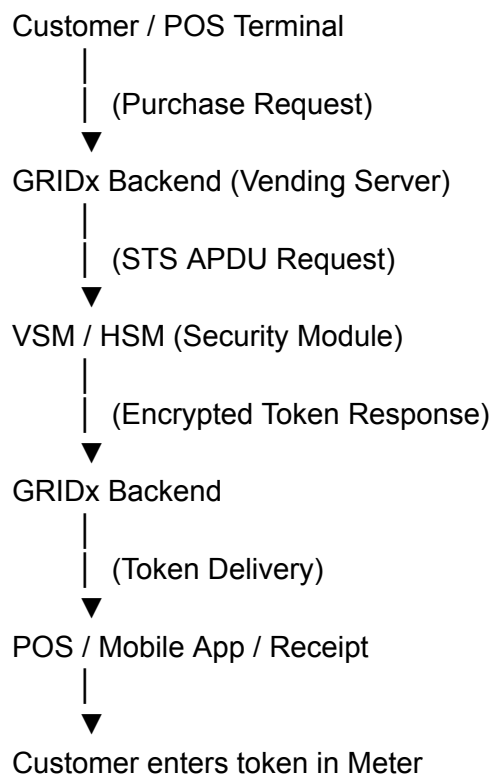
Kamati, I'll show you the **data structure flow between all nodes** in the GRIDx STS vending architecture.

This shows **what data moves between each system component**, from the POS → Backend → VSM/HSM → Meter.

I'll keep it **clear and architecture-level**, because this is exactly what you need when designing the system.

1. System Nodes in GRIDx Architecture

The prepaid vending system typically has these nodes:



Each connection between nodes has **specific data structures**.

2. POS → GRIDx Backend

Purchase Request Structure

This is the **transaction request** from the POS or vending application.

```
struct POS_Vending_Request {  
  
    string meterNumber;    // 11–13 digit meter number  
    string customerID;     // optional customer reference  
    decimal purchaseAmount; // e.g. N$100  
    string vendorID;       // vending agent ID  
    string tariffID;       // tariff applied  
    datetime timestamp;    // transaction time  
  
}
```

Purpose:

This tells the backend:

"Customer with meter X wants to buy N\$100 electricity."

3. GRIDx Backend Internal Data Structure

The backend enriches the request with **meter security information**.

```
struct Backend_Vending_Context {  
  
    string meterNumber;  
    string SGC;           // Supply Group Code  
    int tariffIndex;  
    int keyRevisionNumber; // KRN  
    int keyType;  
    int keyExpiryNumber;  
  
    decimal purchaseAmount;  
    uint32 tokenIdIdentifier; // minutes since base date  
    uint8 randomNumber;  
  
}
```

Purpose:

This structure gathers **all data required for STS token generation**.

4. Backend → VSM/HSM

APDU Token Generation Request

This is the **official IEC 62055-41 request message** sent to the security module.

```
struct APDU_Request {  
  
    char MeterPAN[18];    // meter identifier  
    char TCT[2];          // token carrier type  
    char DKGA[2];         // key generation algorithm  
    char EA[2];           // encryption algorithm  
  
    char SGC[6];          // supply group code  
    char TI[2];           // tariff index  
    char KRN;             // key revision number  
    char KT;              // key type  
    uint8 KEN;            // key expiry number  
  
    char BDT[2];          // base date  
  
    TokenData token;      // 66-bit token structure  
  
}
```

This is the **core message the STS security module understands**.

5. Token Data Structure (Inside APDU)

This is the **66-bit token payload** that gets encrypted.

```
struct TokenData {
```

```
uint8 Class;    // token class
uint8 SubClass; // electricity / water / etc
uint8 RND;      // random value

uint32 TID;     // token identifier
uint16 Amount;  // encoded purchase amount

uint16 CRC;     // data integrity check

}
```

Total size:

66 bits

This payload is **encrypted by the VSM/HSM**.

6. VSM/HSM Internal Processing

Inside the security module:

Select vending key based on SGC

|

Generate decoder key

|

Encrypt TokenData

|

Transpose class bits

|

Generate encrypted token

Output:

Encrypted 66-bit token

7. VSM/HSM → Backend

Token Response Structure

The security module returns a **TCDU response**.

```
struct TCDU_Response {  
  
    uint64 encryptedToken;    // encrypted 66-bit token  
  
    char TCT[2];              // token carrier type  
    char IDRecord[35];        // optional  
    char PRNRecord[];         // optional  
  
}
```

This encrypted token becomes the **20-digit token**.

8. Backend → POS

Final Token Delivery Structure

The backend converts the encrypted data into **customer format**.

```
struct Token_Delivery {  
  
    string meterNumber;  
    string token;           // 20 digit STS token  
    decimal amountPaid;  
    decimal energyUnits;  
  
    string transactionRef;  
    datetime timestamp;  
  
}
```

Example:

Meter: 01234567890
Token: 6234-8901-2345-6789-0123
Amount: N\$100
Energy: 42.7 kWh

9. Customer → Meter

Token Entry

The final step is manual entry into the meter.

Input:

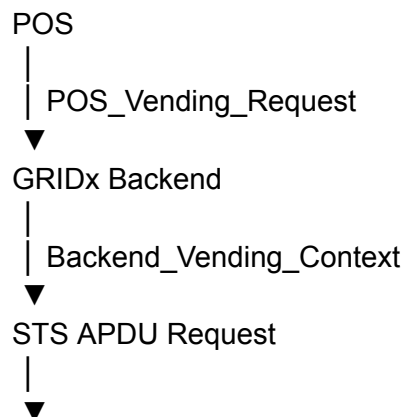
62348901234567890123

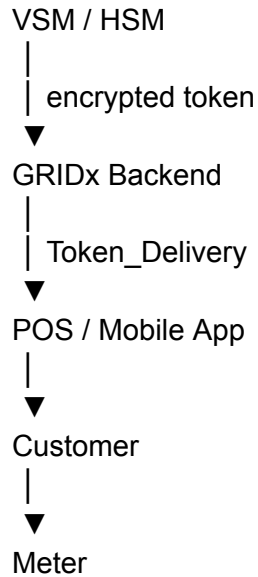
Meter internally:

decrypt token
verify CRC
verify TID
load credit

10. Full GRIDx Data Flow

Here is the **complete structure flow across nodes**:





11. Key Architectural Insight

Your GRIDx system will manage **three different types of data structures**:

Business Data

(customer, payment)

meterNumber
purchaseAmount
vendorID
customerID

STS Security Data

(token generation)

SGC
KRN
KEN
TID
TokenData

Transaction Data

(audit & reporting)

transactionID

token

amount

energy

timestamp

12. Why This Architecture Matters

This layered structure ensures:

- **Security separation** (STS crypto isolated)
- **Scalable vending system**
- **STS certification compliance**

It is exactly how **large vending systems (Eskom, Conlog, Landis+Gyr)** are designed.

If you want, I can also show you the **complete production architecture for GRIDx**, including:

- HSM cluster
- vending API
- mobile vending
- meter management
- key management server

This will show you **the exact architecture used by national utilities** and how you can implement it for Namibia.

Kamati, below is the **same architecture and explanation converted into structured tables**, while **keeping the same numbering and flow** so you can easily use it in documentation or a technical proposal.

1. System Nodes in GRIDx Architecture

Node	Description	Data Flow
Customer / POS Terminal	The operator or vending machine where the electricity purchase begins	Sends purchase request
GRIDx Backend (Vending Server)	Core system that processes transactions, applies tariffs, and prepares STS requests	Communicates with security module
VSM / HSM (Security Module)	Secure cryptographic module that generates STS tokens	Encrypts token data
GRIDx Backend	Receives encrypted token and formats it for delivery	Prepares token for display or receipt
POS / Mobile App	Displays or prints the token	Sends token to customer
Meter	Customer enters the token into the prepaid meter	Meter decrypts token and loads electricity credit

2. POS → GRIDx Backend

Purchase Request Structure

Field	Type	Description
meterNumber	string	Meter identifier (typically 11–13 digits)
customerID	string	Optional customer reference
purchaseAmount	decimal	Amount paid (example N\$100)
vendorID	string	Identifier of vending agent
tariffID	string	Tariff plan used for calculation
timestamp	datetime	Time of transaction

Purpose:

Explanation

This request informs the backend that a specific customer meter wants to purchase a certain amount of electricity credit.

3. GRIDx Backend Internal Data Structure

Field	Type	Description
meterNumber	string	Meter identifier
SGC	string	Supply Group Code
tariffIndex	integer	Tariff block index
keyRevisionNumber	integer	Key Revision Number (KRN)
keyType	integer	Type of security key
keyExpiryNumber	integer	Key expiry identifier
purchaseAmount	decimal	Amount paid
tokenIdIdentifier	uint32	Minutes since STS base date
randomNumber	uint8	Random value used in encryption

Purpose:

Explanation

The backend combines customer purchase data with security information required for STS token generation.

4. Backend → VSM/HSM

APDU Token Generation Request

Field	Size	Description
-------	------	-------------

MeterPAN	18 chars	Meter identifier used by STS
TCT	2 chars	Token Carrier Type
DKGA	2 chars	Decoder Key Generation Algorithm
EA	2 chars	Encryption Algorithm
SGC	6 chars	Supply Group Code
TI	2 chars	Tariff Index
KRN	1 char	Key Revision Number
KT	1 char	Key Type
KEN	8 bits	Key Expiry Number
BDT	2 chars	Base Date indicator
TokenData	66 bits	Token payload structure

Purpose:

Explanation

This message structure is defined by IEC 62055-41 and is the official request format used to generate STS tokens.

5. Token Data Structure (Inside APDU)

Field	Size	Description
Class	2 bits	Token class identifier
SubClasses	4 bits	Type of utility service (electricity, water, gas)
RND	4 bits	Random number for security
TID	24 bits	Token Identifier (minutes since base date)
Amount	16 bits	Encoded electricity value

CRC 16 bits Cyclic Redundancy Check

Total Size:

Total

66 bits

Purpose:

Explanation

This payload contains the transaction information that will be encrypted by the security module to form the prepaid token.

6. VSM/HSM Internal Processing

Step	Description
Select vending key	The system chooses the correct encryption key using the SGC
Generate decoder key	Key derivation process begins
Encrypt TokenData	Token data is encrypted using the STS algorithm
Transpose class bits	Required transformation defined in the STS standard
Generate encrypted token	Final encrypted token is produced
Output:	
Result	
Encrypted 66-bit token	

7. VSM/HSM → Backend

Token Response Structure

Field	Type	Description
encryptedToken	uint64	Encrypted 66-bit token
TCT	char[2]	Token Carrier Type
IDRecord	char[35]	Optional identification information
PRNRecord	variable	Optional printing information

Purpose:

Explanation

The VSM returns the encrypted token data that will later be converted into the 20-digit prepaid token used by the meter.

8. Backend → POS

Final Token Delivery Structure

Field	Type	Description
meterNumber	string	Meter identifier
token	string	20-digit STS token
amountPaid	decimal	Amount paid by customer
energyUnits	decimal	Energy credited to meter
transactionRef	string	Transaction reference ID
timestamp	datetime	Time of vending transaction

Example:

Field	Value
-------	-------

Meter	01234567890
Token	6234-8901-2345-6789-0123
Amount	N\$100
Energy	42.7 kWh

9. Customer → Meter

Token Entry

Step	Description
Customer enters token	Token typed into prepaid meter
Meter decrypts token	Meter uses internal STS keys
Verify CRC	Meter checks token integrity
Verify TID	Prevents token replay
Load credit	Electricity units added

Example Input:

Token
62348901234567890123

10. Full GRIDx Data Flow

Step	Source	Data Structure	Destination
1	POS	POS_Vending_Request	GRIDx Backend

2	Backend	Backend_Vending_Context	Internal processing
3	Backend	APDU Request	VSM / HSM
4	VSM	Encrypted Token	GRIDx Backend
5	Backend	Token_Delivery	POS / Mobile App
6	Customer	Token Entry	Meter

11. Key Architectural Insight

Business Data

Field

meterNumber

purchaseAmount

vendorID

customerID

STS Security Data

Field

SGC

KRN

KEN

TID

TokenData

Transaction Data

Field

transactionID

token

amount

energy

timestamp

12. Why This Architecture Matters

Benefit	Explanation
Security separation	Cryptographic operations isolated in VSM/HSM
Scalable architecture	Supports many POS systems and vendors
STS compliance	Required for certification
Utility-grade reliability	Used in national electricity vending systems

If you'd like, I can also convert this into a **full system architecture diagram for GRIDx** that shows:

- Backend servers
- Token generation service
- HSM cluster
- Vendor POS network
- Smart meter ecosystem

This would look like a **real national prepaid electricity platform design**, similar to what Eskom or large utilities use.